Velammal College of Engineering and Technology,

Viraganoor, Autonomous Madurai – 625 009

Department of Artificial Intelligence and Data Science

VCET Regulation 2021

21AD211 - Computer Networking Principles Laboratory

Lab Manual

Table of contents

Ex. No	Name of the Exercise	Page Number
1	Implement commands like tcpdump, netstat, ifconfig, nslookup and traceroute, ping and traceroute	
2	Implement error correction technique using Hamming Code.	
3	Implementation of socket programs using TCP	
4	Simulation of sliding window protocols	
5	Implementation of ARP/RARP	
6	Implementation of routing protocols	
7	Perform encryption, decryption using the following substitution techniques (i) Ceaser cipher, (ii) Hill Cipher	
8	Implement asymmetric encryption and decryption using RSA Algorithm.	
9	Implement the Diffie-Hellman Key Exchange algorithm for a given problem.	
10	Implement the SIGNATURE SCHEME – Digital Signature Standard.	

Ex. No 1. Implement commands like tcpdump, netstat, ifconfig, nslookup and traceroute, ping and traceroute

Aim: To execute network commands.

1. tcpdump

tcpdump is a command-line packet analyzer that allows users to capture and analyze network traffic.

syntax:

sudo tcpdump -i [interface] [options]

Example:

sudo tcpdump -i eth0 -c 10

This command captures 10 packets on the eth0 interface.

Sample Output:

tcpdump: verbose output suppressed, use -v or -vv for full protocol decode

listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes12:34:56.789012 IP 192.168.1.2.12345 > 192.168.1.1.80: Flags [S], seq 123456789, win 29200, options [mss 1460,sackOK,TS val 123456789 ecr 0,nop,wscale 7], length 0

...

2. netstat

netstat displays network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.

syntax:

netstat [options]

Example:

netstat -tuln

This command lists all listening ports with numeric addresses.

Sample Output:

Active Internet connections (only servers)

Proto Recv-Q Send-Q Local Address Foreign Address State

tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN

udp 0 0 0.0.0.0:68 0.0.0.0:*

3. ifconfig

ifconfig is used to configure network interfaces. It displays information about all active interfaces when used without arguments.

syntax:

ifconfig [interface] [options]

Example:

ifconfig eth0

This command displays the configuration of the eth0 interface.

Sample Output:

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500

inet 192.168.1.2 netmask 255.255.255.0 broadcast 192.168.1.255

inet6 fe80::1a2b:3c4d:5e6f:7g8h prefixlen 64 scopeid 0x20<link>

ether 01:23:45:67:89:ab txqueuelen 1000 (Ethernet)

RX packets 123456 bytes 987654321 (987.6 MB)

RX errors 0 dropped 0 overruns 0 frame 0

TX packets 654321 bytes 123456789 (123.4 MB)

TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

4. nslookup

nslookup queries Internet domain name servers to obtain domain name or IP address mapping.

syntax:

nslookup [domain]

Example:

nslookup example.com

Sample Output:

Server: 8.8.8.8Address: 8.8.8.8#53

Non-authoritative answer:Name: example.comAddress: 93.184.216.34

5. traceroute

traceroute displays the route packets take to a network host.

syntax:

traceroute [destination]

Example:

bash

Copy code

traceroute example.com

Sample Output:

traceroute to example.com (93.184.216.34), 30 hops max, 60 byte packets

1 192.168.1.1 (192.168.1.1) 1.123 ms 1.456 ms 1.789 ms

2 10.0.0.1 (10.0.0.1) 2.123 ms 2.456 ms 2.789 ms

3 93.184.216.34 (93.184.216.34) 3.123 ms 3.456 ms 3.789 ms

6. ping

ping checks the network connectivity between the host and a target by sending ICMP echo requests.

syntax:

ping [destination]

Example:

ping example.com

Sample Output:

PING example.com (93.184.216.34): 56 data bytes64 bytes from 93.184.216.34: icmp_seq=0 ttl=56 time=10.123 ms64 bytes from 93.184.216.34: icmp_seq=1 ttl=56 time=10.456 ms64 bytes from 93.184.216.34: icmp_seq=2 ttl=56 time=10.789 ms

...

Result: Thus the fundamental networking commands are executed.

Ex. No 2. Implement error correction technique using Hamming Code.

Aim: To perform error correction using hamming code.

Hamming code algorithm:

Hamming Code is an error-detecting and error-correcting code used to ensure data integrity in communication systems. It can detect and correct single-bit errors.

Algorithm Steps

Input Data

1. Start with a binary data string of length **m**.

Calculate Redundant Bits (r)

1. Find the number of redundant bits required using the formula: 2r≥m+r+12^r \geq m + r + 12r≥m+r+1 Where:
    1. **m** = number of data bits
    2. **r** = number of redundant bits

Position the Redundant Bits

1. Place the redundant bits in positions that are powers of 2 (e.g., 1, 2, 4, 8...).

Calculate Parity for Redundant Bits

1. Compute parity values for each redundant bit by checking parity for corresponding positions.

Insert Parity Bits

1. Replace the redundant bit positions with computed parity values.

Transmit the Codeword

1. Send the complete codeword (data + redundant bits) to the receiver.

Error Detection at Receiver

1. Recalculate parity bits at the receiver end.
2. Identify the position of the error (if any) based on parity calculations.

Error Correction

1. If error position is non-zero, flip the erroneous bit to correct it.

Program:

```
#include <stdio.h>
#include <math.h>
// Function to calculate the parity bits
void calculateParityBits(int data[], int n) {
    int r = 0, m = n;
    // Calculate the number of parity bits
    while ((m + r + 1) > pow(2, r)) {
        r++;
    }
    int hammingCode[m + r + 1];

    // Insert data bits into the Hamming code
    int j = 0, k = 0;
    for (int i = 1; i <= m + r; i++) {
        if ((i & (i - 1)) == 0) {
            hammingCode[i] = 0; // Place for parity bits
        } else {
            hammingCode[i] = data[j++];
        }
    }
    // Calculate parity bits
    for (int i = 1; i <= m + r; i++) {
```

```
        if ((i & (i - 1)) == 0) {
            int parity = 0;
            for (int k = i; k <= m + r; k++) {
                if ((k & i) == i) {
                    parity ^= hammingCode[k];
                }
            }
            hammingCode[i] = parity;
        }
    }
    printf("Encoded Hamming Code: ");
    for (int i = 1; i <= m + r; i++) {
        printf("%d", hammingCode[i]);
    }
    printf("\n");
}
int main() {
    int n;
    printf("Enter the number of data bits: ");
    scanf("%d", &n);
    int data[n];
    printf("Enter the data bits: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data[i]);
    }
    calculateParityBits(data, n);
    return 0;
}
```

OUTPUT

```
Enter the number of data bits: 4
Enter the data bits:  1 0 1 1
Encoded Hamming Code: 0110011
```

Result: Thus the hamming code generated for verifying the data bits.

Ex. No 3. Implementation of socket program using TCP

Aim: To create sockets using TCP for getting date and time.

Algorithm for TCP Socket Programming

1. Server Side Algorithm

Create a Socket

1. Use the socket() function to create a socket.
2. Specify AF_INET for IPv4 and SOCK_STREAM for TCP.

Bind the Socket

Bind the socket to a specific IP address and port number using the bind() function.

Listen for Connections

Use the listen() function to wait for incoming client connections.

Accept Connection

Use the accept() function to accept a client connection request.

Send/Receive Data

Use send() and recv() functions to exchange data with the connected client.

Close the Connection

Use the close() function to terminate the connection.

---

2. Client Side Algorithm

Create a Socket

Use the socket() function to create a socket.

onnect to Server

Use the connect() function to establish a connection with the server.

Send/Receive Data

Use send() and recv() functions to exchange data with the server.

Close the Connection

Use the close() function to terminate the connection.

Program:
<u>Client program:</u>

```
#include<stdio.h>

#include<string.h>

#include<stdlib.h>

#include<sys/types.h>

#include<sys/socket.h>

#include<netdb.h>

int main()

{

int sd,c;
```

```
char msg[50],sip[15];
struct sockaddr_in caddr;
struct hostent *he;
printf("Enter the ip address: ");
scanf("%s", sip);
he= gethostbyname(sip);
sd = socket(AF_INET,SOCK_STREAM,0);
if(sd!=-1)
{
printf("Socket is created");
}
else
{
printf("socket is not created");
}
caddr.sin_family = AF_INET;
caddr.sin_addr   = *((struct in_addr*)he->h_addr);
caddr.sin_port   = htons(1200);
c= connect(sd, (struct sockaddr*)&caddr,sizeof(caddr));
if(c==0)
printf("connected to server");
else
printf("not connected");
recv(sd,msg,sizeof(msg),0);
printf(" The day and time : %s",msg);
close(sd);
```

```
return 0;
}
```

Server program:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netdb.h>
int main()
{
int sd,b,cd;
char msg[100];
struct sockaddr_in caddr,saddr;
socklen_t clen=sizeof(caddr);
time_t tick;
sd=socket(AF_INET,SOCK_STREAM,0);
if(sd!=-1)
printf("\nsocket is created");
else
printf("\nsocket is not created");
saddr.sin_addr.s_addr=htons(INADDR_ANY);
saddr.sin_port=htons(1200);
b=bind(sd,(struct sockaddr*)&saddr,sizeof(saddr));
if(b==0)
printf("\nbinded to client");
```

```
else
printf("\nnot binded");
listen(sd,5);
while(1);
{
cd=accept(sd,(struct sockaddr*)&caddr,&clen);
tick=time(NULL);
printf("Sys day & time: %s",ctime(&tick));
strcpy(msg,(char*)ctime(&tick));
send(cd,msg,sizeof(msg),0);
}
close(sd);
close(cd);
return 0;
}
```

Output

* To get output from server the server code have to be executed first.*\

TCP SERVER

```
[sys45@localhost ~]$ cc tcpserver.c
[sys45@localhost ~]$ ./a.out tcpserver
```

****

socket is created

binded to client

Sys day & time: Mon Sep 6 15:41:50 2010

****

TCP CLIENT

```
[sys45@localhost ~]$ cc tcpclient.c

[sys45@localhost ~]$ ./a.out tcpclient.c

*****************************************************

Enter the ip address: 192.168.1.104

Socket is created

connected to server

The day and time : Mon Sep  6 15:45:58 2024

*****************************************************
```

Result: Thus the program for socket programming and client server communication is executed.

Ex. No 4. Simulation of sliding window protocol

Aim: To implement flow control using sliding window protocol.

Sliding Window Protocols

Sliding window protocols are used in data link and transport layers to ensure reliable and efficient transmission of data. Two main types are:

1. **Go-Back-N (GBN)**
2. **Selective Repeat (SR)**

1. Go-Back-N (GBN) Protocol

Algorithm

Sender Side:

1. **Initialize variables:**
    1. Set base = 0 and next_seq_num = 0.
    2. Define window_size.
2. **Send packets:**

    1. Send packets within the window (base to base + window_size - 1).
    2. Increment next_seq_num.

3. **Wait for Acknowledgment:**

    1. Start a timer for the oldest unacknowledged packet.
    2. If acknowledgment (ACK) is received for packet n, slide the window by setting base = n + 1.

4. **Timeout Handling:**

    1. If timeout occurs, **retransmit all packets** starting from base.

5. **Repeat Steps 2–4** until all data is sent.

Receiver Side:

1. **Receive a packet:**

    1. If the packet is in order, send ACK and deliver data to the upper layer.
    2. If the packet is out of order, **discard it** and resend ACK for the last in-order packet.

2. **Repeat** until all packets are received.

2. Selective Repeat (SR) Protocol

Algorithm

Sender Side:

1. **Initialize variables:**

    - Set base = 0 and next_seq_num = 0.
    - Define window_size.

2. **Send packets:**

    - Send packets within the window (base to base + window_size - 1).
    - Increment next_seq_num.

3. **Wait for Acknowledgment:**

    - If ACK is received for packet n, mark it as acknowledged.
    - Slide the window forward only if consecutive packets are acknowledged.

4. **Timeout Handling:**
    - If timeout occurs for a specific packet, **retransmit only that packet**.
5. **Repeat Steps 2–4** until all data is sent.

Receiver Side:

1. **Receive a packet:**
    - If the packet is in order, deliver it and send ACK.
    - If the packet is out of order, buffer it and send ACK.
2. **Deliver buffered packets in sequence.**
3. **Repeat** until all packets are received.

Program:

```
#include<stdio.h>

#include<stdlib.h>

#include<math.h>

int n,r;

struct frame
{

char ack;

int data;

}frm[10];

int sender(void);

void recvfrm(void);

void resend(void);
void resend1(void);

void goback(void);

void selective(void);

int  main()
```

```
{
int c;
do
{
printf("\n 1.selective repeat arq\n 2.go back arq \n 3.exit");
printf("\n enter choice");
scanf("%d",&c);
switch(c)
{
case 1:
selective();
break;
case 2:
goback();
break;
case 3:
exit(0);
break;
}
}
while(c!=4);
}
void goback()
{
sender();
recvfrm();
resend1();
printf("\n all packets sent");
```

```
}
void selective()
{
sender();
recvfrm();
resend();
printf("\n all packets are sent");
}
int sender()
{
int i;
printf("\n enter the number of packets to be sent");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("\n enter data fr packet[%d]",i);
scanf("%d",&frm[i].data);
frm[i].ack='y';
}
return 0;
}
void recvfrm()
{
int i;
rand();
r=rand()%n;
frm[r].ack='n';
for(i=0;i<n;i++)
```

```
{
if (frm[i].ack=='n')
{
printf("\n packet no %d is not recieved",r);
}
}
}
void resend()
{
printf("\n resending packet %d",r);
sleep(2);
frm[r].ack='y';
printf("\n recieved packet is %d",frm[r].data);
}
void resend1()
{
int i;
printf("\n resending packet from %d ",r);
for(i=r;i<n;i++)
{
sleep(2);
frm[i].ack='y';
printf("\n recieved data of packet %d is %d",i,frm[i].data);
}
}
```

output-sliding window

```
[student08@localhost 08]$ gcc sliding.c
[student08@localhost 08]$ ./a.out
```

```
1.selective repeat arq
2.go back arq
3.exit
enter choice1
enter the number of packets to be sent2
enter data fr packet[0]3
enter data fr packet[1]4
packet no 0 is not received
resending packet 0
received packet is 3
all packets are sent
1.selective repeat arq
2.go back arq
3.exit
enter choice2
enter the number of packets to be sent3
enter data fr packet[0]1
enter data fr packet[1]2
enter data fr packet[2]3
packet no 1 is not received
resending packet from 1
received data of packet 1 is 2
received data of packet 2 is 3
all packets sent
1.selective repeat arq
2.go back arq
3.exit
enter choice3
```

Result: Thus the program for sliding window protocol is executed.

Ex. No 5. Implementation of ARP/RARP

Aim: To resolve the MAC (Media Access Control) address corresponding to a given IP (Internet Protocol) address in a network using ARP.

Algorithm:

Step 1: Broadcast ARP Request

1. The sender (host) prepares an **ARP Request** packet.
2. The packet contains the sender's IP and MAC addresses, along with the target's IP address.
3. The sender sets the target MAC address as **unknown** (all zeros).
4. The ARP Request is broadcasted to all devices in the network using the **broadcast MAC address (FF:FF:FF:FF:FF:FF)**.

Step 2: Receive ARP Request

1. All devices in the network receive the ARP Request packet.
2. Each device compares the target IP address in the packet with its own IP address.

Step 3: Send ARP Reply

If the target IP matches the receiver's IP address:

1. The receiver prepares an **ARP Reply** packet containing its **MAC address**.

2. Sends this ARP Reply **directly (unicast)** to the sender.

If the target IP does **not** match, the device ignores the ARP Request.

Step 4: Update ARP Cache

1. The sender receives the ARP Reply and updates its **ARP cache/table** with the resolved MAC address.
2. Stores the IP-to-MAC mapping for future communication.

Step 5: Data Transmission

1. The sender uses the resolved MAC address to send data packets to the target device.
2. If the MAC address is not found in the ARP cache later, the process repeats from Step 1.

Program - ARP

```
#include <stdio.h>
#include <string.h>

// Structure to represent an ARP Table Entry
struct ARPEntry {
   char ip_address[16];  // IPv4 address as a string
   char mac_address[18]; // MAC address as a string
};

// ARP Table (Static Entries)
struct ARPEntry arp_table[] = {
   {"192.168.1.1", "00:0a:95:9d:68:16"},
   {"192.168.1.2", "00:0b:22:44:55:66"},
   {"192.168.1.3", "00:0c:33:77:88:99"},
   {"192.168.1.4", "00:0d:44:aa:bb:cc"}
};

// Size of ARP Table
int table_size = sizeof(arp_table) / sizeof(arp_table[0]);

// Function to find MAC address for a given IP address
void find_mac_address(char ip_address[]) {
   int found = 0;

   for (int i = 0; i < table_size; i++) {
      if (strcmp(arp_table[i].ip_address, ip_address) == 0) {
         printf("IP Address: %s --> MAC Address: %s\n", ip_address, arp_table[i].mac_address);
         found = 1;
         break;
```

```
        }
    }

    if (!found) {
        printf("IP Address: %s not found in ARP Table.\n", ip_address);
    }
}

// Main Function
int main() {
    char ip_address[16];

    // Input IP address to resolve
    printf("Enter IP Address to resolve: ");
    scanf("%s", ip_address);

    // Find and display corresponding MAC address
    find_mac_address(ip_address);

    return 0;
}
```

OUTPUT:
Enter IP Address to resolve: 192.168.1.1
IP Address: 192.168.1.1 --> MAC Address: 00:0a:95:9d:68:16

Aim: To resolve the IP address corresponding to a given MAC address in a network.

Algorithm:

Broadcast RARP Request:

1. The client machine, which knows its MAC address but does not know its IP address, sends a **RARP Request** as a **broadcast packet**.
2. It specifies its **MAC address** in the request.

Receive RARP Request:

1. The RARP Server listens for the incoming request.
2. It checks its database for an entry matching the **MAC address** received in the request.

Send RARP Reply:

1. If the MAC address is found in the database, the server sends a **unicast RARP Reply** containing the corresponding **IP address** to the client.
2. If not found, the server ignores the request.

Assign IP Address:

1. The client receives the RARP Reply and configures the assigned **IP address** for further communication.

Program:

```
#include <stdio.h>
#include <string.h>

// Structure to store RARP Table Entry
struct RARPEntry {
   char mac_address[18]; // MAC Address
   char ip_address[16];  // IP Address
};

// RARP Table (Static entries)
struct RARPEntry rarp_table[] = {
   {"00:0a:95:9d:68:16", "192.168.1.1"},
   {"00:0b:22:44:55:66", "192.168.1.2"},
   {"00:0c:33:77:88:99", "192.168.1.3"},
   {"00:0d:44:aa:bb:cc", "192.168.1.4"}
};

// Size of RARP Table
int table_size = sizeof(rarp_table) / sizeof(rarp_table[0]);

// Function to find IP address for a given MAC address
void find_ip_address(char mac_address[]) {
   int found = 0;

   for (int i = 0; i < table_size; i++) {
      if (strcmp(rarp_table[i].mac_address, mac_address) == 0) {
         printf("MAC Address: %s --> IP Address: %s\n", mac_address, rarp_table[i].ip_address);
         found = 1;
         break;
      }
   }

   if (!found) {
      printf("MAC Address: %s not found in RARP Table.\n", mac_address);
   }
}

// Main Function
int main() {
   char mac_address[18];

   // Input MAC address
```

```
    printf("Enter MAC Address to resolve: ");
    scanf("%s", mac_address);

    // Find and display corresponding IP address
    find_ip_address(mac_address);

    return 0;
}
```

Output:
Enter MAC Address to resolve: 00:0a:95:9d:68:16
MAC Address: 00:0a:95:9d:68:16 --> IP Address: 192.168.1.1

Result: Thus the IP address and MAC address is resolved using ARP/RARP protocol.

Ex. No 6. Implementation of Routing Protocol - OSPF

Aim: To simulate the OSPF protocol that uses Dijkstra's algorithm to compute the shortest path in a network.

Algorithm:

Program:

```
#include <stdio.h>
#include <limits.h> // For INT_MAX
#include <stdbool.h> // For boolean values

#define V 6 // Number of vertices
// Function to find the vertex with minimum distance
int minDistance(int dist[], bool visited[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
```

```
}
// Function to print the shortest distance and path
void printSolution(int dist[]) {
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < V; i++)
        printf("%d\t%d\n", i, dist[i]);
}
// Dijkstra's algorithm for OSPF
void dijkstra(int graph[V][V], int src) {
    int dist[V];     // Distance from source to each vertex
    bool visited[V]; // To track visited vertices
    // Initialize distances and visited status
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }
    // Distance from source to itself is always 0
    dist[src] = 0;
    // Process all vertices
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited); // Get the minimum distance vertex
        visited[u] = true; // Mark vertex as processed
        // Update distance for neighbors
        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    // Print the result
    printSolution(dist);
}
// Main function
int main() {
    // Graph represented as adjacency matrix
    int graph[V][V] = {
        {0, 10, 0, 0, 0, 11},
        {10, 0, 18, 0, 0, 0},
        {0, 18, 0, 6, 0, 0},
        {0, 0, 6, 0, 9, 0},
        {0, 0, 0, 9, 0, 4},
```

```
    {11, 0, 0, 0, 4, 0}
  };
  int source = 0; // Set source vertex
  // Apply OSPF algorithm
  dijkstra(graph, source);
  return 0;
}
```

Output:

Vertex	Distance from Source
0	0
1	10
2	28
3	24
4	15
5	11

Result: Thus the shortest path is found using OSPF routing protocol.

Ex. No 7. Encryption and Decryption using the substitution techniques.

AIM:

To encrypt and decrypt the given message by using Ceaser Cipher encryption algorithm.

ALGORITHMS:

1. In Ceaser Cipher each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet.
2. For example, with a **left shift of 3**, **D** would be replaced by **A**, **E** would become **B**, and so on.
3. The encryption can also be represented using modular arithmetic by first transforming the letters into numbers, according to the scheme, **A = 0, B = 1, Z = 25.**
4. Encryption of a letter x by a shift n can be described mathematically as,

En(x) = (x + n) mod26

5. Decryption is performed similarly,

Dn (x)=(x - n) mod26

PROGRAM:

CaesarCipher.java

```
class caesarCipher {
public static String encode(String enc, int offset) {
offset = offset % 26 + 26;
StringBuilder encoded = new StringBuilder();
for (char i : enc.toCharArray()) {
if (Character.isLetter(i)) {
if (Character.isUpperCase(i)) {
encoded.append((char) ('A' + (i - 'A' + offset) % 26));
} else {
encoded.append((char) ('a' + (i - 'a' + offset) % 26));
}
} else {
encoded.append(i);}
}
}
return encoded.toString();
}
public static String decode(String enc, int offset) {
return encode(enc, 26 - offset);
}
public static void main(String[] args) throws java.lang.Exception {
String msg = "Anna University";
```

```
System.out.println("Simulating Caesar Cipher\n
");
System.out.println("Input : " + msg);
System.out.printf("Encrypted Message : ");
System.out.println(caesarCipher.encode(msg, 3));
System.out.printf("Decrypted Message : ");
System.out.println(caesarCipher.decode(caesarCipher.encode(msg, 3), 3));
}
}
```

OUTPUT:

```
Simulating Caesar Cipher
Input : Anna University
Encrypted Message : Dqqd
Xqlyhuvlwb Decrypted Message :
Anna University
```

RESULT:

Thus the program for ceaser cipher encryption and decryption algorithm hasbeen implemented and the output verified successfully.

AIM:

To implement a program to encrypt and decrypt using the Hill cipher substitution technique.

ALGORITHM:

1. In the Hill cipher Each letter is represented by a number modulo 26.

2. To encrypt a message, each block of n letters is multiplied by an invertible ***n x n*** matrix, again ***modulus 26***.

3. To decrypt the message, each block is multiplied by the inverse of the matrix used for encryption.

4. The matrix used for encryption is the cipher key, and it should be chosen randomly from the ***set of invertible n × n matrices (modulo 26).***

5. The cipher can, be adapted to an alphabet with any number of letters.

6. All arithmetic just needs to be done modulo the number of letters instead of modulo 26.

PROGRAM:

HillCipher.java

```
class hillCipher {

/* 3x3 key matrix for 3 characters at once */

public static int[][] keymat = new int[][] { { 1, 2, 1 }, { 2, 3, 2 },

{ 2, 2, 1 } }; /* key inverse matrix */

public static int[][] invkeymat = new int[][] { { -1, 0, 1 }, { 2, -1, 0 }, { -2, 2, -1

} };

public static String key = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

private static String encode(char a, char b, char c)

{ String ret = "";
```

```
int x, y, z;

int posa = (int) a - 65;

int posb = (int) b - 65;

int posc = (int) c - 65;

x = posa * keymat[0][0] + posb * keymat[1][0] + posc * keymat[2][0];

y = posa * keymat[0][1] + posb * keymat[1][1] + posc * keymat[2][1];z = posa * keymat[0][2] +
posb * keymat[1][2] + posc * keymat[2][2];

a = key.charAt(x % 26);

b = key.charAt(y % 26);c = key.charAt(z % 26);

ret = "" + a + b + c;

return ret;

}

private static String decode(char a, char b, char c)

{ String ret = "";

int x, y, z;

int posa = (int) a - 65;

int posb = (int) b - 65;

int posc = (int) c - 65;

x = posa * invkeymat[0][0] + posb * invkeymat[1][0] + posc *

invkeymat[2][0];

y = posa * invkeymat[0][1] + posb * invkeymat[1][1] + posc *

invkeymat[2][1];

z = posa * invkeymat[0][2] + posb * invkeymat[1][2] + posc *

invkeymat[2][2];

a = key.charAt((x % 26 < 0) ? (26 + x % 26) : (x % 26));

b = key.charAt((y % 26 < 0) ? (26 + y % 26) : (y % 26));

c = key.charAt((z % 26 < 0) ? (26 + z % 26) : (z % 26));
```

```
ret = "" + a + b + c;
return ret;
}
public static void main(String[] args) throws java.lang.Exception
{ String msg;
String enc =
""; String dec
= ""; int n;
msg = ("SecurityLaboratory");
System.out.println("simulation of Hill Cipher\n
");
System.out.println("Input message : " + msg);
msg = msg.toUpperCase();
msg = msg.replaceAll("\\s", "");
/* remove spaces */ n = msg.length() % 3;
/* append padding text X */ if (n != 0) {
for (int i = 1; i <= (3 - n); i++) {
msg += 'X';
}}
System.out.println("padded message : " + msg);
char[] pdchars = msg.toCharArray();
for (int i = 0; i < msg.length(); i += 3) {
enc += encode(pdchars[i], pdchars[i + 1], pdchars[i + 2]);
}
System.out.println("encoded message : " + enc);
char[] dechars = enc.toCharArray();
```

```
for (int i = 0; i < enc.length(); i += 3) {
dec += decode(dechars[i], dechars[i + 1], dechars[i + 2]);
}
System.out.println("decoded message : " + dec);
}
}
```

OUTPUT:

Simulating Hill Cipher

Input Message : SecurityLaboratory

Padded Message : SECURITYLABORATORY

Encrypted Message : EACSDKLCAEFQDUKSXU

Decrypted Message : SECURITYLABORATORY

RESULT:

Thus the program for hill cipher encryption and decryption algorithm has been implemented and the output verified successfully.

Ex. No 8. Implement asymmetric encryption and decryption using RSA algorithm

AIM:

To implement RSA (Rivest–Shamir–Adleman) algorithm by using HTML and Javascript.

ALGORITHM:

1. Choose two prime number p and q
2. Compute the value of n and **p**

3. Find the value of ***e*** (public key)

4. Compute the value of ***d*** (private key) using gcd()

5. Do the encryption and decryption

a. Encryption is given as,

c = t e mod n

b. Decryption is given as,

t = c d mod n

PROGRAM:

rsa.html

```
<html>
<head>
<title>RSA Encryption</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<center>
<h1>RSA Algorithm</h1>
<h2>Implemented Using HTML & Javascript</h2>
<hr>
<table>
<tr>
<td>Enter First Prime Number:</td><td><input type="number" value="53" id="p"></td>
</tr>
<tr>
<td>Enter Second Prime Number:</td>
```

```
<td><input type="number" value="59" id="q"></p></td>
</tr>
<tr>
<td>Enter the Message(cipher text):<br>[A=1, B=2,...]</td>
<td><input type="number" value="89" id="msg"></p>
</td>
</tr>
<tr>
<td>Public Key:</td>
<td>
<p id="publickey"></p>
</td>
</tr>
<tr>
<td>Exponent:</td>
<td>
<p id="exponent"></p>
</td>
</tr>
<tr>
<td>Private Key:</td>
<td>
<p id="privatekey"></p>
</td>
</tr>
<tr>
```

```
<td>Cipher Text:</td>
<td>
<p id="ciphertext"></p>
</td>
</tr>
<tr>
<td><button onclick="RSA();">Apply RSA</button></td>
</tr>
</table></center>
</body>
<script type="text/javascript">
function RSA() {
var gcd, p, q, no, n, t, e, i, x;gcd = function (a, b) { return (!b) ? a : gcd(b, a % b); };
p = document.getElementById('p').value;
q = document.getElementById('q').value;
no = document.getElementById('msg').value;
n = p * q;
t = (p - 1) * (q - 1);
for (e = 2; e < t; e++) {
if (gcd(e, t) == 1) {
break;
}
}
for (i = 0; i < 10; i++) {
x = 1 + i * t
if (x % e == 0) {
```

```
d = x / e;
break;
}
}
ctt = Math.pow(no, e).toFixed(0);
ct = ctt % n;
dtt = Math.pow(ct, d).toFixed(0);
dt = dtt % n;
document.getElementById('publickey').innerHTML = n;
document.getElementById('exponent').innerHTML = e;
document.getElementById('privatekey').innerHTML = d;
document.getElementById('ciphertext').innerHTML = ct;
}
</script>
</html>
```

OUTPUT:

RSA Algorithm

Implemented Using HTML & Javascript

Enter First Prime Number:	53
Enter Second Prime Number:	59
Enter the Message(cipher text): [A=1, B=2,...]	89
Public Key:	3127
Exponent:	3
Private Key:	2011
Cipher Text:	1394

Apply RSA

RESULT:

Thus the RSA algorithm has been implemented using HTML & CSS and the output has been verified successfully.

Ex. No 9. Implement the Diffie-Hellman Key Exchange algorithm for a given problem.

AIM:

To implement the Diffie-Hellman Key Exchange algorithm for a given problem.

ALGORITHM:

1. Alice and Bob publicly agree to use a modulus $p = 23$ and base $g = 5$

(which is a primitive root modulo 23).

2. Alice chooses a secret integer $a = 4$, then sends Bob $A = g^a \bmod p$

$A = 5^4 \bmod 23 = 4$

3. Bob chooses a secret integer $b = 3$, then sends Alice $B = g^b \bmod p$

$B = 5^3 \bmod 23 = 10$

4. Alice computes $s = B^a \bmod p$

$s = 10^4 \bmod 23 = 18$

5. Bob computes $s = A^b \bmod p$

$s = 4^3 \bmod 23 = 18$

5. Alice and Bob now share a secret (the number 18).

PROGRAM:

DiffieHellman.java

```
class DiffieHellman {

public static void main(String args[]) {

int p = 23; /* publicly known (prime number) */

int g = 5; /* publicly known (primitive root) */

int x = 4; /* only Alice knows this secret */

int y = 3; /* only Bob knows this secret */

double aliceSends = (Math.pow(g, x)) % p;

double bobComputes = (Math.pow(aliceSends, y)) % p;

double bobSends = (Math.pow(g, y)) % p;

double aliceComputes = (Math.pow(bobSends, x)) % p;

double sharedSecret = (Math.pow(g, (x * y))) % p;

System.out.println("simulation of Diffie-Hellman key exchange algorithm\n--

");

System.out.println("Alice Sends : " + aliceSends);

System.out.println("Bob Computes : " + bobComputes);System.out.println("Bob Sends : " +
bobSends);

System.out.println("Alice Computes : " + aliceComputes);

System.out.println("Shared Secret : " + sharedSecret);

/* shared secrets should match and equality is transitive */

if ((aliceComputes == sharedSecret) && (aliceComputes == bobComputes))

System.out.println("Success: Shared Secrets Matches! " + sharedSecret);

else

System.out.println("Error: Shared Secrets does not Match");

}

}
```

OUTPUT:

simulation of Diffie-Hellman key exchange algorithm

Alice Sends : 4.0

Bob Computes :

18.0 Bob Sends :

10.0

Alice Computes : 18.0

Shared Secret : 18.0

Success: Shared Secrets Matches! 18.0

RESULT:

Thus the *Diffie-Hellman key exchange algorithm* has been implemented using Java Program and the output has been verified successfully.

Ex. No 10. Implement the SIGNATURE SCHEME – Digital Signature Standard.

Aim: To implement signature verification using DSS .

Description:

Key Pair Generation:

1. Uses **DSA (Digital Signature Algorithm)** to generate **Public** and **Private Keys**.
2. Key size is set to **2048 bits** for higher security.

Signature Generation:

1. Creates a digital signature using the **SHA-256 hashing algorithm** combined with **DSA**.
2. Signs the given data using the **Private Key**.

Signature Verification:

1. Verifies the authenticity of the data by comparing it against the **Public Key**.

Program:

```
import java.security.*;
import java.util.Base64;
public class DigitalSignatureSystem {
   // Method to generate Key Pair
   public static KeyPair generateKeyPair() throws NoSuchAlgorithmException {
      KeyPairGenerator keyGen = KeyPairGenerator.getInstance("DSA"); // Using DSA
Algorithm
      keyGen.initialize(2048); // Key size
      return keyGen.generateKeyPair();
   }
   // Method to generate Digital Signature
   public static byte[] generateSignature(String data, PrivateKey privateKey) throws Exception {
      Signature signature = Signature.getInstance("SHA256withDSA");
      signature.initSign(privateKey); // Initialize with Private Key
      signature.update(data.getBytes()); // Update with data
      return signature.sign(); // Generate Signature
   }
   // Method to verify Digital Signature
   public static boolean verifySignature(String data, byte[] signatureBytes, PublicKey publicKey)
throws Exception {
      Signature signature = Signature.getInstance("SHA256withDSA");
      signature.initVerify(publicKey); // Initialize with Public Key
```

```
        signature.update(data.getBytes()); // Update with data
        return signature.verify(signatureBytes); // Verify the Signature
    }
    public static void main(String[] args) {
        try {
            // Generate Key Pair
            KeyPair keyPair = generateKeyPair();
            // Data to be signed
            String data = "This is a sample message to be signed.";
            // Generate Signature
            byte[] digitalSignature = generateSignature(data, keyPair.getPrivate());
            // Display Results
            System.out.println("Original Data: " + data);
            System.out.println("Digital Signature: " +
              Base64.getEncoder().encodeToString(digitalSignature));
            // Verify Signature
            boolean isVerified = verifySignature(data, digitalSignature, keyPair.getPublic());
            System.out.println("Signature Verification: " + (isVerified ? "Valid" : "Invalid"));

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Original Data: This is a sample message to be signed.

Digital Signature: QWEiX42k3Rz/qPnuy77scB3t==

Signature Verification: Valid

Result:

Thus the program for DSS is executed.